

Gaussian splatting integrációja Godot környezetben

Mérnökinformatikus MSc
Vizuális informatika főspecializáció
Diplomatervezés 1.

Gaussian splatting

Gauss-eloszlású
ellipszoid-alakú
splat-ok halmaza



Cél

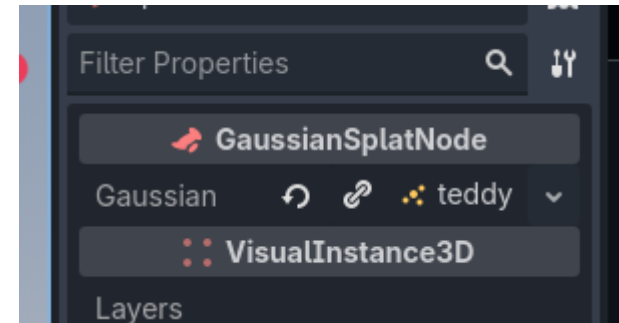
- Godot játékmotorba történő integrálás
- interaktív játék készítésének lehetősége splat-os modellekkel
- egyszerre splat-os modellek, és háromszöghálós modellek megjelenítése
- könnyen integrálható editor-addon
- újralightolható objektumok

Önlab 1 alatt elvégzett munka

- GodotGaussianSplatting + VitaVehicle projektekkel demó, ehhez:
 - Egyszerre több pontfelhő megjelenítése
 - A pontfelhők különböző transzformációja
 - Automata objektum-megtalálás
 - Mélységhelyes összekombinálás
 - Projektek összekombinálása
 - Adatok beolvasása másik formátumból
 - Kitérő: Mesh2Splat linux portja

Elsőosztályú polgárok

- Editor ikonok
- Betöltő script:
 - így már exportált projektekkel is könnyebben működik
- Editor ikont a témának megfelelően színezi az editor
- AMD bug kijavítása: a projekt már AMD kártyákon is használható

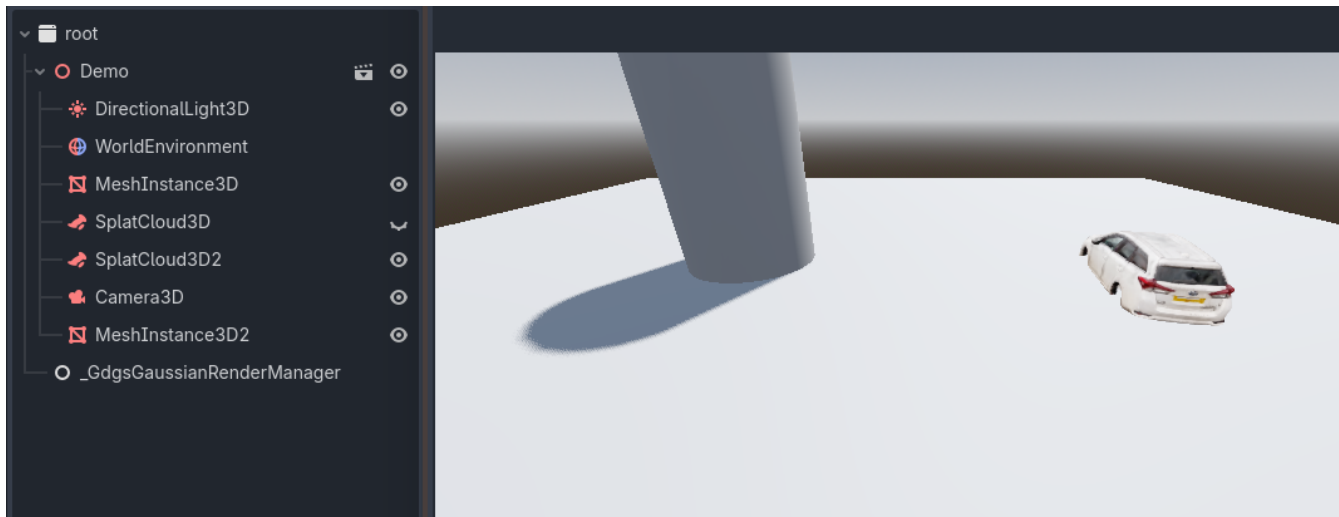


CompositorEffect, projektek merge-elése

- Fullscreen Quad vs CompositorEffect a splates megjelenítendő kép sima pipeline-ba integrálásához
- Új implementáció is megjelent: úgy döntöttem, hogy egyesítsük a projekteket, főbb feature-öket átvettük egymástól
- Ő tőlem: covariancia mátrix, amd bugfix, editor ikonok, dinamikus láthatóság, instancing
- Én tőle: CompositorEffect, jobb blending, architektúra, stb.

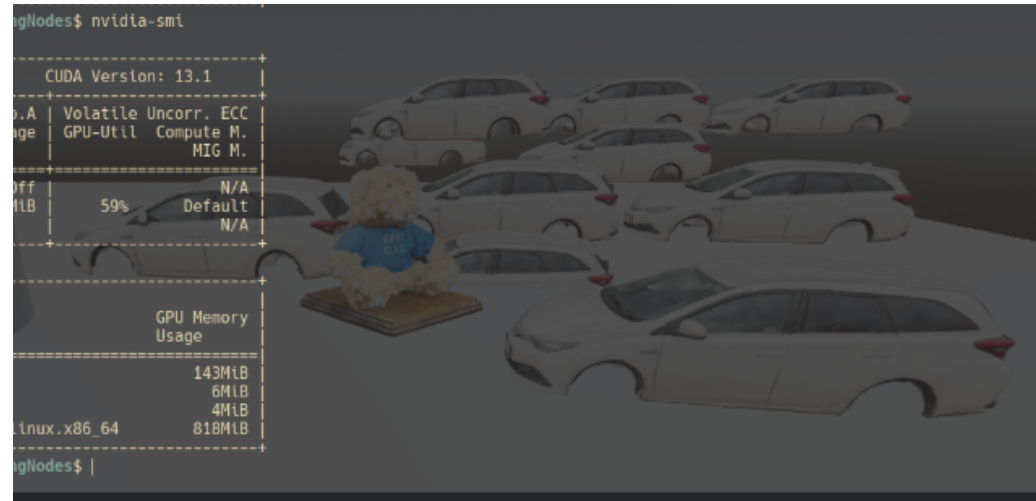
Dinamikus láthatóság

- Visible toggle hatására
 - eltűnjön / megjelenjen a pontfelhő
 - ezt gyorsan csinálja, felesleges másolások nélkül
- Végül: transzformációs mátrix mellé egy bool



Instancing

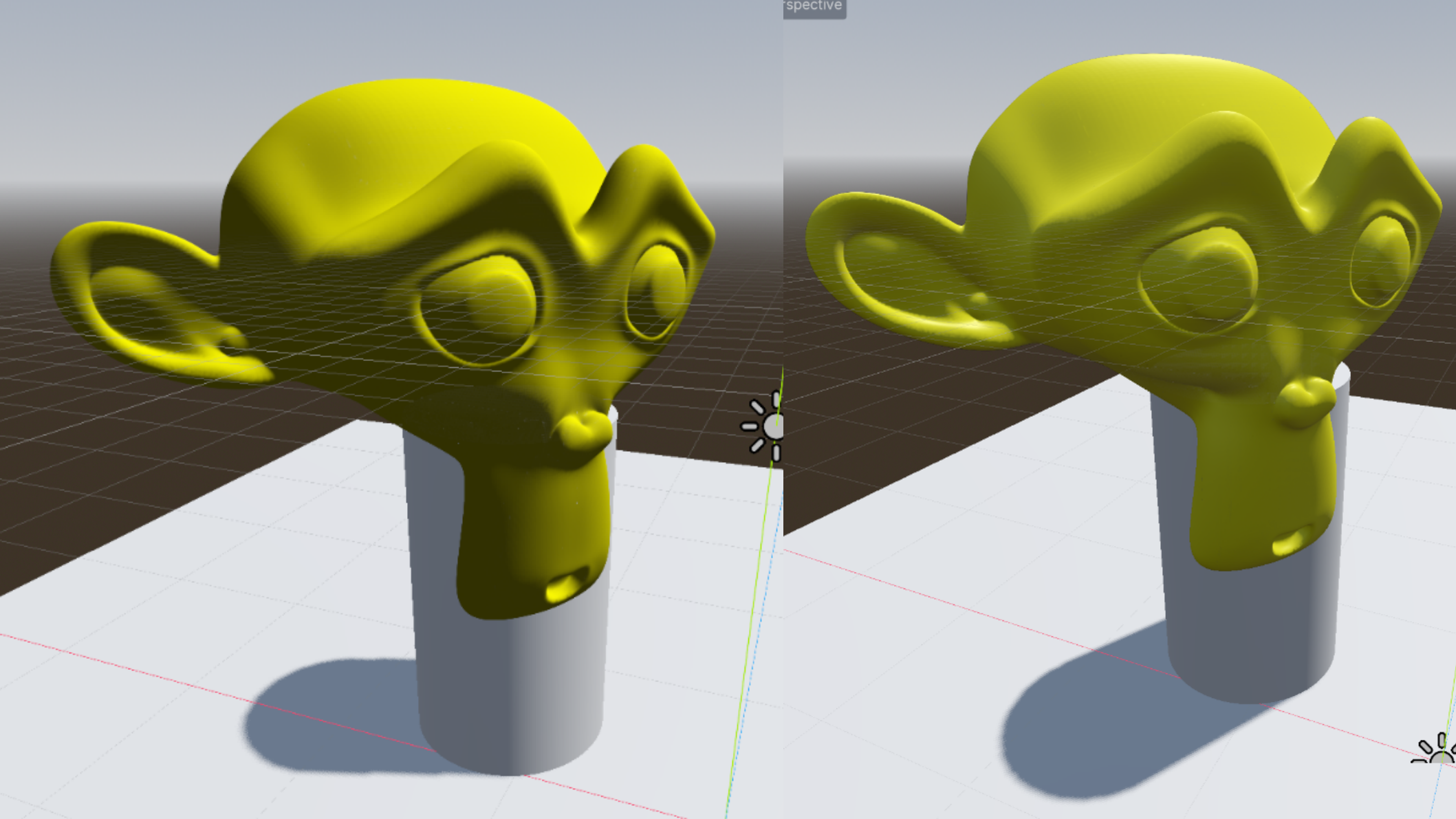
- Fontos, külön szerepel a feladatkiírásban
- Sima Splat – ID és
ID → Transzformáció helyett:
- ID1 – ID2
ID1 → Splat
ID2 → Transzformáció
- Így: egy splat pont(és így a pontfelhő is) többször kirajzolható, más-más transzformációval
- Kb. 10 autó modellel: 1207MiB vs 818 MiB felhasznált memória



Relightolás

- Szintén fontos, külön szerepel a feladatkiírásban
- 2-féle módszer (saját shading, vagy a Godot deferred shading-je)
- Mindkettőnél előnyök, hátrányok
 - Saját shading:
 - előny: CompositorEffect-tel használható, átlátszóság működik
 - hátrány: össze kell gyűjteni manuálisan a fényeket, és nekünk kell shade-elni
 - Godot deferred shading:
 - előny: pont úgy fogja az engine shade-elni, mint a scene többi részét, automatikus pontfényforrás, spotlight támogatás (elvileg legalábbis)
 - hátrány: muszáj az FSQuados megoldást használni, átlátszóság nem működik normálisan
- Mesh2Splat: már fordul linuxon, sajnos nem az én PR-emet merge-elték, de talán felhasználták az újhoz. Kell a könnyű PBR-es pontfelhők generálásához

perspective



Kitérő: Typst

- Dokumentum szövegének írásához a Typst mellett döntöttem
- Modern LaTeX alternatíva
- Szobatársam (Szarkowicz Dániel) ajánlására
- Elég kényelmes

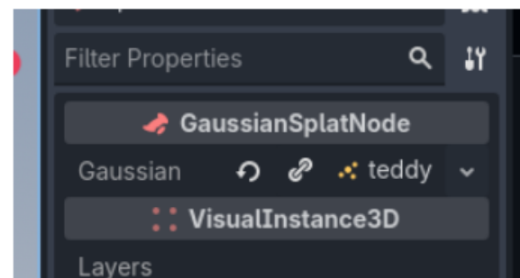
Gaussian splatting integrációja

Godot környezetben

DIPLOMATERV

Készítette
Vértessaljai Bálint

Konzulens
Dr. Vaitkus Márton



3. ábra: A Node és a Resource saját ikonokat kaptak

GaussianSplatNode	GaussianResource
Piros, átfedő pöttyök	Sárga, kisebb sugarú pöttyök

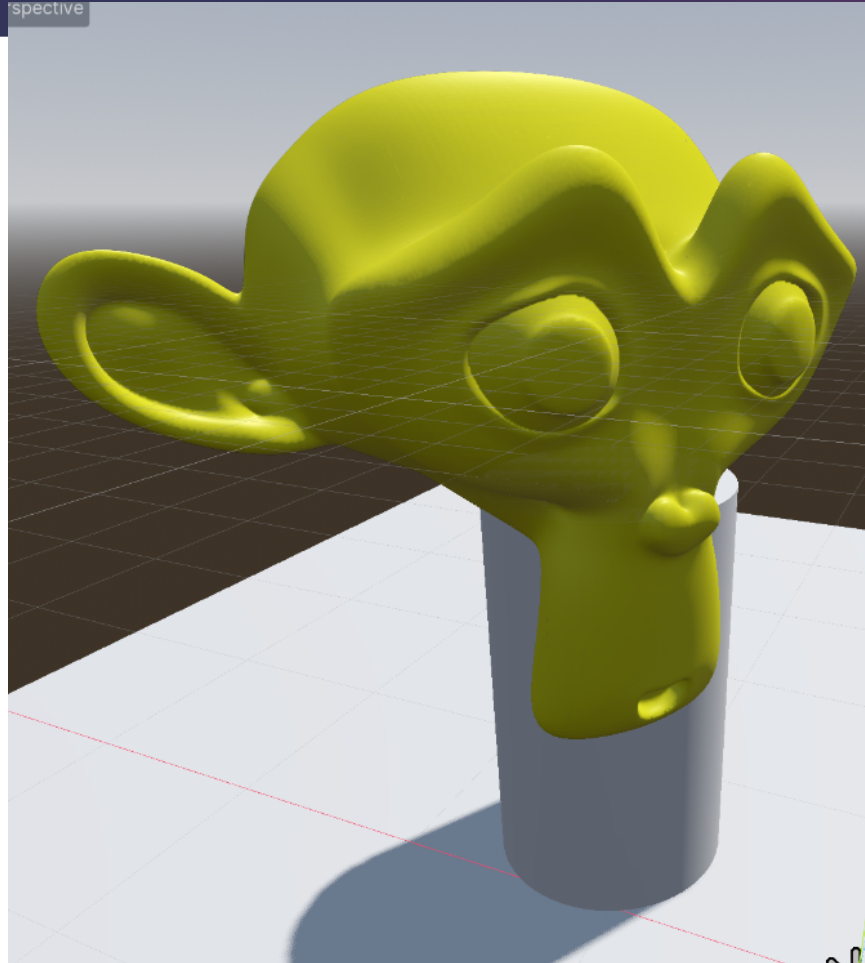
1. táblázat: Táblázat teszt

4.5.3. AMD Bug

A tesztelések során figyelmes lettem egy AMD hardveren történő bug-ra, ezt a VulkanRadixSort könyvtárra vezettem vissza, és az eltérő subgroup méret okozhatta. A hibát én is kijavítottam, és láttam azt is, hogy upstream is kijavították már, csak az általam használt eredeti megjelenítő nem frissítette a dependency-t. Ezt jeleztem az ő repositoryjukban is, a megfelelő fixet megjelölve [6]. A linkelt issue-ban találhatóak a további linkek a kapcsolódó fixekre.

Köszönöm szépen a figyelmet!

- Kérdések?



Extra dia: Gaussian splatting előnyei

- képek alapján könnyebben rekonstruálható
- fotorealisztikus minőségű rekonstrukció lehetséges, nézeti irány is figyelembe vehető
- GPU-n renderelhető valós időben

